

AWS DEVOPS INTERVIEW GUIDE

Scenario 1: Website is Hosted on EC2 Behind ALB – How Will You Configure Route 53?

Scenario

Your company has deployed an application on EC2 instances behind an Application Load Balancer. Users should access the application using **www.company.com** instead of the ALB DNS name.

How would you configure Route 53?

Solution

Architecture

```
Internet
|
Route53 Hosted Zone
|
Alias Record
|
Application Load Balancer
|
EC2 Instances
```

Step 1: Register or Transfer Domain

Example:

`company.com`

Can be purchased from

- Route53
 - GoDaddy
 - Namecheap
-

Step 2: Create Hosted Zone

Hosted Zone:
company.com

Route53 generates

NS Records
SOA Record

Step 3: Configure DNS

Create an Alias Record.

Record Type:
A

Alias:
Yes

Target:
Application Load Balancer

Why Alias Instead of CNAME?

AWS recommends Alias because

- Works for root domain
 - Free DNS queries
 - Automatically updates ALB IP changes
-

Step 4: Verify

dig company.com

nslookup company.com

Should return ALB IPs.

Interview Points

✓ Alias Record

✓ Hosted Zone

✓ NS Records

✓ SOA Record

✓ ALB Integration

Scenario 2: Multi-Region Disaster Recovery Using Route 53

Scenario

Your application is running in

Mumbai

and

Singapore

If Mumbai fails, traffic should automatically move to Singapore.

How will you configure it?

Solution

Use

Failover Routing Policy

Architecture

Route53

Primary → Mumbai ALB

Secondary → Singapore ALB

Configure Health Checks

Health Check

<https://app.company.com/health>

Every

30 sec

If unhealthy



Route53 sends traffic to

Singapore

DNS Records

Primary

A Alias

Failover:
Primary

Secondary

A Alias

Failover:
Secondary

Result

Mumbai Healthy



Traffic → Mumbai

If

Mumbai Down



Traffic → Singapore

Benefits

- Automatic failover
 - No manual intervention
 - Minimal downtime
-

Scenario 3: Users from India Should Access Mumbai Server and US Users Should Access Virginia Server

Scenario

Company has two deployments:

- Mumbai
- Virginia

Indian users should use Mumbai.

US users should use Virginia.

Solution

Use

Geolocation Routing

Architecture

India

↓

Mumbai ALB

US

↓

Virginia ALB

DNS Configuration

Record 1

India

→ Mumbai

Record 2

US

→ Virginia

Default

→ Mumbai

Benefits

- Lower latency
 - Better performance
 - Regional compliance
-

Difference from Latency Routing

Geolocation

Country Based

Latency

Actual AWS latency

Scenario 4: How Will You Perform Blue-Green Deployment Using Route 53?

Scenario

Production is running Version 1.

Version 2 is deployed.

You want gradual migration.

Solution

Use

Weighted Routing

Initial State

Blue

100%

Green

0%

After Testing

Blue

80%

Green

20%

Later

Blue

50%

Green

50%

Finally

Blue

0%

Green

100%

Advantages

- Zero downtime
 - Easy rollback
 - Progressive rollout
-

Interview Tip

If Green has issues

Weight = 0

Traffic instantly returns to Blue.

Scenario 5: Customers Report Website is Slow from Different Countries

Scenario

Users worldwide complain that DNS resolves slowly.

How would Route 53 improve performance?

Solution

Use

Latency-Based Routing

Architecture

Tokyo

Virginia

Mumbai

London

Route53 measures latency and routes to the AWS Region with the **lowest network latency**, not necessarily the closest geographically.

Example

User

Germany

May be routed to

London

because it has lower latency.

Benefits

- Better user experience
 - Reduced response time
 - Automatic latency optimization
-

Important Routing Policies to Remember

Routing Policy	Use Case
Simple	Single resource
Weighted	Blue-Green, Canary deployment
Latency	Lowest network latency
Geolocation	Country/Region-based routing
Failover	Disaster Recovery
Multi-Value Answer	Return multiple healthy endpoints

AWS CloudFront – Top 5 Scenario-Based Interview Questions with Detailed Answers

Scenario 1: Static Website Hosted in S3 is Slow for Global Users

Scenario

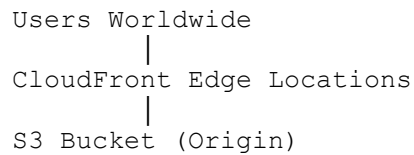
Your company hosts a static website in an S3 bucket. Users from Europe, Asia, and North America report slow page load times.

How would you improve performance?

Solution

Use

Architecture



CloudFront caches static content (HTML, CSS, JS, images) at edge locations close to users.

Steps

1. Create an S3 bucket and enable static website hosting (or use the bucket as a private origin).
 2. Create a CloudFront distribution.
 3. Set the S3 bucket as the origin.
 4. Configure the default cache behavior.
 5. Update DNS (Route 53 Alias record) to point to the CloudFront distribution.
-

Benefits

- Reduced latency
 - Lower load on S3
 - Global content delivery
 - HTTPS support
-

Scenario 2: Protect an S3 Bucket So It Can Only Be Accessed Through CloudFront

Scenario

Your application stores images in S3, but users should not access the bucket directly. All requests must go through CloudFront.

How would you implement this?

Solution

Use

CloudFront + Origin Access Control (OAC)

(Previously, Origin Access Identity (OAI) was commonly used.)

Architecture

```
Users
|
CloudFront
|
OAC
|
Private S3 Bucket
```

Steps

1. Make the S3 bucket private.
 2. Create an Origin Access Control (OAC).
 3. Attach the OAC to the CloudFront origin.
 4. Update the S3 bucket policy to allow only the CloudFront distribution to read objects.
-

Benefits

- No public S3 access
 - Improved security
 - Content served only through CloudFront
-

Scenario 3: Your Website Content Changes Frequently, but Users Still See the Old Version

Scenario

After deploying a new version of your website, users continue seeing outdated images or HTML files.

How would you resolve this?

Solution

This is caused by cached content at CloudFront edge locations.

Two common solutions:

Option 1: CloudFront Invalidation

Invalidate specific paths:

```
/index.html  
/images/*
```

or all files:

```
/*
```

CloudFront removes cached copies and fetches the latest content from the origin.

Option 2: Versioned File Names (Recommended)

Instead of:

```
app.js
```

Use:

```
app-v2.js
```

or

```
app.45ab23.js
```

This avoids frequent invalidations and improves caching efficiency.

Best Practice

- Version static assets
 - Invalidate only when necessary
 - Set appropriate Cache-Control headers
-

Scenario 4: Deliver Different Content Based on User Location

Scenario

Users in Europe should receive European content, while users in Asia should receive localized content.

How would CloudFront help?

Solution

Use

CloudFront Geographic Restrictions

or configure cache behaviors and use headers (such as CloudFront viewer country headers) at the application or edge function level.

Example

```
Europe Users
  |
CloudFront
  |
EU Content
Asia Users
  |
CloudFront
  |
Asia Content
```

You can use:

- CloudFront Functions
 - Lambda@Edge
 - Application logic based on viewer country headers
-

Benefits

- Localized content
- Better user experience
- Regional customization

Scenario 5: Dynamic API Responses Are Not Updating Correctly

Scenario

Your frontend is served through CloudFront, and API requests are also routed through CloudFront. Users receive stale API responses.

How would you fix this?

Solution

Separate caching behavior for static and dynamic content.

Example Behaviors

`/images/*`

Cache aggressively.

`/css/*`

Cache aggressively.

`/js/*`

Cache aggressively.

`/api/*`

Disable caching or set a very low TTL, and forward the required headers, cookies, and query strings to the origin as needed.

Benefits

- Fresh API responses
 - Fast static asset delivery
 - Improved overall application performance
-

CloudFront Features Frequently Asked in Interviews

Feature	Purpose
Edge Locations	Cache content close to users
Origin	Source of content (S3, ALB, EC2, API Gateway, etc.)
Distribution	CloudFront configuration for content delivery
Cache Behavior	Defines caching rules for URL patterns
TTL	Controls how long content stays cached
Invalidation	Removes cached content before TTL expires
Origin Access Control (OAC)	Secure private S3 access through CloudFront
Signed URLs/Cookies	Restrict access to private content
HTTPS	Encrypts communication between users and CloudFront
CloudFront Functions / Lambda@Edge	Execute logic at edge locations for request/response customization